

RESEARCH NOTEBOOK

Jev & System One Models

What a typed-decision model changes in AI systems, and what it doesn't

SUBJECT	<i>AI systems · reliability · latency · cost</i>
OPENED	<i>21 September 2026</i>
SOURCES	<i>TypeSafe primary docs + 5 independent evals</i>
STATUS	<i>Fact-checked · post drafted</i>

"Types can guarantee a valid answer. Not a right one."

Contents

1	Executive summary	15 findings	3
2	What Jev actually is	input · output · API	4
3	Architecture vs LLMs	why no decode loop	7
4	RLCD & calibration		9
5	Benchmark audit	every headline number	10
6	Failure modes		13
7	Implications for agents		14
8	Three post angles		16
9	Recommended angle		17
10	The LinkedIn post	849 words	18
11	Fact-check table		20
12	Sources		22

BOTTOM LINE

Jev is a closed-set decision model sold as an API. Its typed interface removes malformed outputs, not wrong answers. The headline multiples are TypeSafe's own, measured against slow, reasoning-heavy setups. First-week independent tests show real but smaller gains and uneven calibration.

How to read the margin stamps

- V

 verified in a primary source or by independent measurement
- VR

 vendor-reported: TypeSafe's own claim or measurement
- I

 my inference or arithmetic
- O

 opinion

Numbers come from pages opened on 21 Sep 2026. "Independent" evals are single-author GitHub projects, not peer-reviewed; they're labelled as such wherever they're used.

1 Executive summary

Fifteen things worth knowing, most important first-ish.

1. *What it is.* TypeSafe AI launched Jev on 15 Sep 2026. Text or JSON state plus typed questions go in; probabilities come out, never prose. Founder Diogo Almeida is a co-author of the InstructGPT paper. (V)
2. *Three primitives.* Noul returns P(yes). Choice picks one of up to 255 options with a distribution and confidence. Score rates 2-10 ordered levels and returns an expected level plus confidence. (V)
3. *Architecture is undisclosed.* TypeSafe cites a new architecture, a "parallel sampler" and a training method called RLCD. No paper, parameter count or recipe is public. TechCrunch reports a transformer trained on synthetic data. (VR)
4. *"Can't hallucinate" = "can't return an out-of-schema value."* TypeSafe's FAQ concedes Jev can pick the wrong option. The launch post says its 0% hallucination figure is by construction, not measured. (V)
5. *Schema validity isn't unique to Jev.* Strict structured-output modes on major LLM APIs already constrain decoding to the schema (except refusals and token limits). Jev's real differentiators: native per-option probabilities, many questions per pass, input-only pricing. (V) (I)
6. *193.6x faster, 444.6x cheaper* comes from TypeSafe's own four-workflow eval. Reference answers are the averaged outputs of GPT-6 Astra and Claude Fable 5.1, not human labels. TypeSafe staff built the workflows; TypeSafe calls the multiples the high end. (VR)
7. *The multiple depends on the comparator.* Jev scores 67.8% vs GPT-5.6 Terra's 67.9%. From the published per-case means that's ~76x cheaper and ~25x faster than Terra, and ~8x cheaper than GPT-5.6 Luna (66.8%). (I)
8. *The accuracy gap is task-shaped.* Near the top on customer service (76.0% vs best 78.3%); weakest on invoice processing (61.8% vs 79.1%), the multi-document reconciliation task. (VR)
9. *The latency claim broadly holds.* Independent setups: 239 ms p50 from France, ~430 ms floor via OpenRouter, ~0.43 s via Vercel AI Gateway. Against small non-reasoning LLMs the speedup is ~2-3x, not 200x. (V)
10. *Below frontier on direct questions.* Banking77: Jev 83.2%, Terra 87.5%, supervised encoder 93.3%. CLINC150: Jev 87.0%, Terra 91.5%, gpt-5.4-nano 79.5%. Phishing verdict: Jev 62.6%, Claude Haiku 4.5 81.3%. (V*)
11. *Decomposition is the lever.* Five narrow phishing signals from the same call, combined by a cross-validated logistic regression, reached 95.1%. Supervised models trained on labels still beat zero-shot Jev where tested. (V*)

the key reframe!

vendor eval,
reference = other
models

12. *Calibration is the core claim.* I found no calibration curve or ECE published by TypeSafe. One independent study: ECE ~ 0.03 on public benchmarks, 0.107 out of distribution; Choice overconfident, Noul underconfident. (V*)
13. *Closed-world failure replaces free-form hallucination.* Without a "none of these" option, 0 of 30 out-of-scope messages were flagged, even at a 0.99 gate. With the deciding policy absent from the input: right 44.7% of the time at 0.74 average probability. (V*)
14. *TypeSafe documents its own limits.* Its jev-1.13 page lists literal reading, counting and math, dates, multi-hop indirection, irrelevant state, adversarial content, inconsistent invariants, and no generation. (VR)
15. *Pricing.* \$0.042 per million input tokens, output unbilled, 64k tokens per request. TypeSafe says it can't prove the price isn't subsidized. Testers confirm tiny bills. (V)

least evidenced claim

* single-author GitHub evaluations, not peer-reviewed; one model version (jev-1.13), one setup each.

2 What Jev actually is

A function call with a judgment inside it: state + typed questions in, one probability distribution per question out. Your code, not the model, decides what to do with each distribution.

Input

- **State** is a string, JSON object or array, text only. "Unstructured state" in TypeSafe's framing means program data that carries text: a ticket, a log excerpt, a record with free-text fields. Not a chat transcript.
- **Questions** are a named map. Each has a type, *instructions*, and optional *criteria*: option descriptions (Choice), ordered level descriptions (Score), or true/false descriptions (Noul).
- **Limits**: 64k tokens per request, 32k for state + longest question, up to 255 Choice options, 2-10 Score levels, 1,200 requests/min. English is most reliable; no images or audio.
- Every question is evaluated in parallel and in isolation against the same state. TypeSafe says adding questions barely changes latency.

Output

The three primitives

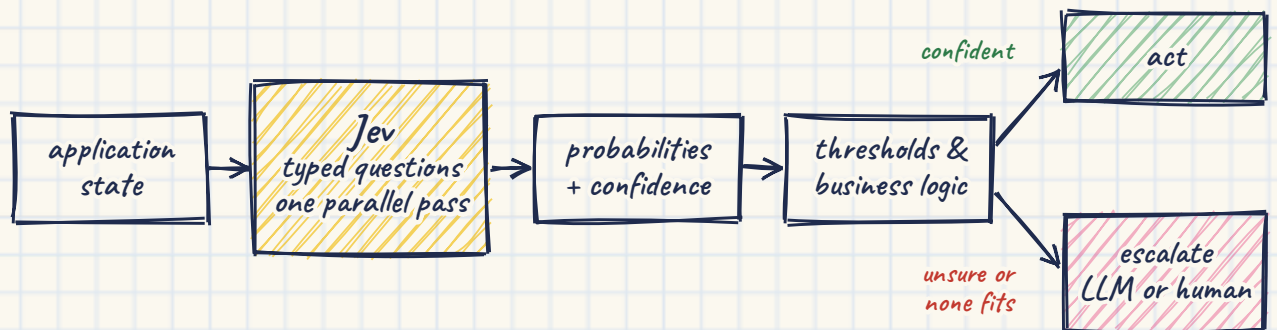
[docs.typesafe.ai / primitives](https://docs.typesafe.ai/primitives)

Primitive	Returns	Confidence field
Noul	<code>noul</code> = P(yes), 0–1	None: the probability is the whole answer
Choice	<code>choice</code> + <code>probabilities</code> over every option	Yes, derived from how peaked the distribution is
Score	<code>score</code> = $\sum \text{level} \times p(\text{level})$, <code>probabilities</code> , <code>legend</code>	Yes, same derivation

TypeSafe's docs describe *confidence* as a summary of the returned distribution, not a separate model output. Their demo approximates it, for a 3-option Choice, as $(3 \times \text{largest probability} - 1) / 2$.

Output tokens are counted but not billed; a call returns a handful of numbers per question.

Confidence isn't a second opinion. It's a statistic of the same distribution: 1.0 when all mass sits on one option. Miscalibrated probabilities = miscalibrated confidence.



the model's job ends at the distribution; routing, weighting and escalation live in ordinary code

A realistic call

Python, shape from TypeSafe's quick start

ticket, thresholds and helpers are illustrative,

not an observed run

```

from typesafe_sdk import Choice, Noul, Score, TypeSafeClient

client = TypeSafeClient(model="jev-latest")

r = client.system_one(
    state={"ticket": "Charged twice for order #4411. Need this fixed today.",
          "customer_tier": "enterprise"},
    questions={
        "queue": Choice(
            instructions="Which team should handle this ticket?",
            criteria={
                "billing": "Payments, refunds, invoices",
                "technical": "Bugs, integrations, outages",
                "none_of_these": "Fits none of the queues above", # the safety valve
            },
        ),
        "urgency": Score(
            instructions="How time-sensitive is the request?",
            criteria=["No deadline", "Wants it soon", "Explicit same-day deadline"],
        ),
        "asks_refund": Noul(instructions="Is the customer asking for money back?"),
    },
)

q = r.answers["queue"]
if q.choice == "none_of_these" or q.confidence < 0.8: # tune on your own labels
    escalate_to_human(r)
else:
    route(q.choice, priority=r.answers["urgency"].score)

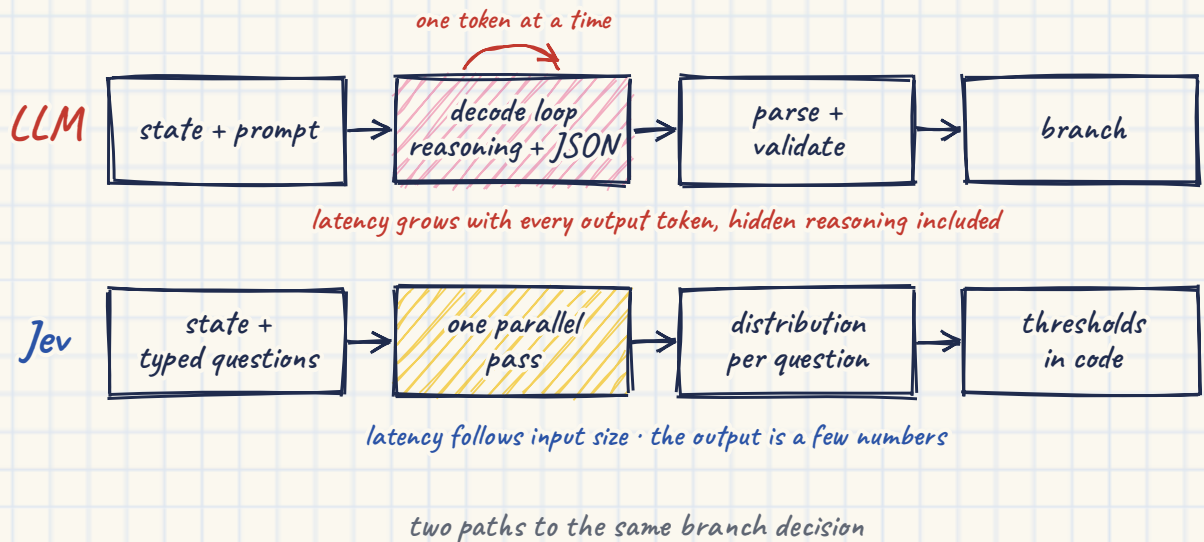
```

What it cannot do

Per TypeSafe's own docs: no free text, no code, no explanations of its reasoning, no multimodal input. Extraction of free values only works by turning candidates into a Choice (e.g. a regex finds candidate dates, Jev picks one). (V)

3 Architecture: Jev vs LLMs

The fair comparison isn't Jev vs "LLMs". It's Jev vs an LLM with strict structured outputs, which already fixes schema validity. Jev's remaining edge is the missing decode loop and native probabilities.



The sketch flatters Jev: its path hides the work that moved upstream into question design, decomposition and threshold calibration. It flatters the LLM path too: with strict structured outputs, parse-and-validate mostly disappears.

diagram flatters both sides!

Side by side

Dimension	Generative LLM (free text)	LLM + strict structured outputs	Jev
Output	Any string	Schema-valid JSON	Distribution over predefined options
Invalid output?	Possible	Largely eliminated by constrained decoding	Impossible by construction
Per-option probabilities	Rarely exposed; verbalized ones poorly calibrated	Often unavailable, especially with hidden reasoning	Always returned
Reasoning first	Yes	Yes	No explicit reasoning step
Latency driver	Output length incl. hidden reasoning	Same	Input length
Cost driver	Input + output (output ~5× pricier, per TypeSafe)	Same	Input only, \$0.042 / M tokens
Flexibility	Anything	Any schema, open values allowed	Bounded answer spaces only
Typical failure	Malformed, off-task or fabricated content	Valid JSON with a wrong or invented value	A valid option that is wrong

Why no decode loop can be much faster (I)

My reading of the mechanism. TypeSafe hasn't published the architecture or the "parallel sampler".

1. An autoregressive model produces **one token per sequential forward pass**. Decoding is memory-bandwidth-bound: each token costs roughly a full read of the weights.
2. Reasoning models emit hundreds to thousands of hidden tokens before answering. Latency, and tail latency, grow with a length the caller doesn't control.
3. A closed-set decision needs **no decode loop**. The answer is a distribution over K known options, readable after one pass over the input.
4. That pass (prefill) is parallel across tokens and GPU-friendly, and many questions can share the state's encoding. TypeSafe reports batching 13 questions into one call was 12.2× cheaper and 10.0× faster, answers unchanged. (VR)
5. Output is a handful of numbers, so it costs almost nothing to serve. "Free output tokens" follows from the architecture; it isn't a discount.

Not new in principle: label-likelihood scoring, cross-encoders and BERT heads never decode either. What's new is the product: runtime questions, many per pass, trained for calibration, one API. (0)

Reach for a structured-output LLM when...

- The decision needs reasoning first: multi-hop logic, reconciling documents, arithmetic. Jev's weakest TypeSafe workflow is invoice processing (61.8% vs 79.1% for GPT-5.6 Sol).
- The label set is open or unknown, or you need free values extracted.
- You need an explanation for audit or a human reviewer.
- The input is images, audio or a long mixed document.
- An LLM call is already in the path and the decision is a byproduct.
- Volume is low and the accuracy gap (2–17 points across TypeSafe's four workflows) costs more than tokens.

Jev's shape fits when...

- Decisions are high-volume over a known answer space.
- Many questions concern the same state, so one pass answers all of them.
- A latency-critical gate sits in front of expensive calls: routing, guardrails, relevance filters.
- You grade or verify at scale, where per-call cost decides whether you check everything or sample.

4 RLCD & calibration

RLCD is TypeSafe's name for its post-training method, not an established technique. The goal is sound; the evidence so far is mostly other people's.

What TypeSafe says (VR)

- **Objective:** Reinforcement Learning for Calibrated Decisions trains the model to return decisions with probabilities that match how often they're right. TypeSafe calls these "epistemically honest" probabilities.
- **vs RLHF:** RLHF optimizes for what raters prefer; TypeSafe argues this rewards sycophancy and confident-sounding answers and causes mode dropping.
- **vs RLVR:** verifiable rewards produced reasoning models, strong at math and code but slower and costlier. RLCD targets fast, bounded decisions.
- **Not published:** the reward, the data, the architecture, any calibration metric. TechCrunch reports the training data is synthetic, made in-house.

What "calibrated" means here

Across many predictions, outcomes given probability 0.8 should happen about 80% of the time. TypeSafe's docs say explicitly this describes groups of predictions, never a single answer. Calibration (ECE, reliability curves) is separate from discrimination: does confidence rank right answers above wrong ones (AUROC)?

calibration is not discrimination. cascades need both

Established context

- **Post-training can damage calibration.** In the GPT-4 technical report (Fig. 8), ECE went from 0.007 for the pre-trained model to 0.074 after post-training. That supports TypeSafe's premise. (V)
- A natural training signal for honest probabilities is a **proper scoring rule** (log loss, Brier), minimized only by reporting true beliefs. Whether RLCD uses one isn't public. (I)
- Miscalibration is often fixable after the fact with **temperature scaling** on held-out labels (Guo et al., 2017). That's also the practical fix for Jev on your data.

What independent tests measured

single-author GitHub evals · jev-1.13 · one setup each

Study (2026)	Setting	Calibration result
jev-ood-calibration · 19 Sep	OpenBookQA, CommonsenseQA, HellaSwag	ECE 0.024 / 0.032 / 0.029. Good, but likely close to training data
same	900 synthetic support tickets (out of distribution)	ECE 0.107 vs a 0.024 noise floor. Choice overconfident (refit T = 3.29); Noul underconfident (T = 0.66)
same	Priority rule depending on a policy absent from the text	44.7% accuracy at 0.74 average probability on the pick
jev-phishing-bench · 17 Sep	2,000 phishing / legit emails	ECE 0.154 vs 0.097 for Claude Haiku 4.5; AUROC 0.689 vs 0.837
jev-baselines-eval · 18 Sep	Banking77, CLINC150	Error-ranking AUROC beat gpt-5.4-nano on one set, lost on the other. Confidence exactly 1.0 on 102/200 CLINC items, 6 of them wrong
priorbench/jev · 20 Sep	400-item classification task	Accuracy above threshold flat from 0.50 to 0.95, then 100% at 0.99, covering 60.2% of traffic

Reading

Calibration looks good on familiar benchmarks and inconsistent out of distribution and across question types. Don't carry a Noul threshold over to a Choice; TypeSafe's own docs warn against it too. 0

Rule of thumb: use confidence as a gate at the extremes (0.99 and up), not as a literal probability in the middle. Recalibrate per question type on your own labels.

5 Benchmark audit

Latency and price claims hold up. The headline multiples are real numbers from a favorable comparison. Nobody has independently reproduced TypeSafe's workflow evals as of 21 Sep 2026.

Claim by claim

Claim	Measured by	Workload & method	Caveats	Independent check
193.6x faster, 444.6x cheaper	TypeSafe	4 workflows; time & cost per case; LLMs at default reasoning	Reference = average of GPT-6 Astra & Fable 5.1 at high thinking; workflows by TypeSafe staff; comparator not stated; "high end" per TypeSafe	not reproduced
70–500 ms end to end	TypeSafe	Evals run from staff laptops on the US West Coast, near the service	Network distance favors TypeSafe	broadly holds 239 ms p50 (France), ~430 ms floor (OpenRouter), ~0.43 s (Vercel)

Claim	Measured by	Workload & method	Caveats	Independent check
40–200× faster than frontier	TypeSafe	"System One shaped" queries vs 3–329 s frontier times (third-party site)	Compared against reasoning models	~2–3× vs small non-reasoning LLMs
\$0.042/M input, free output	Price list	—	TypeSafe can't prove it isn't subsidized	confirmed 4,621 calls cost ~\$0.06
No type errors, zero hallucinations	TypeSafe	By construction; LLM rates in its chart from OpenRouter data	TypeSafe notes the 0% isn't empirical and the LLM data is biased	holds for format 0 invalid in 4,621 calls; 0 failed gradings in 10,500
Calibrated probabilities	TypeSafe	Asserted	No curve or ECE published that I could find	mixed see section 4
~68% accuracy, near mid-tier LLMs	TypeSafe	Agreement with frontier consensus	Agreement, not correctness	not reproduced

The multiple depends on the comparator ^(I)

Computed from TypeSafe's published per-case means (Jev: 67.8%, \$0.0004, 0.4 s). Jev's cost and time are rounded on the site, so treat these as $\pm 10\text{--}15\%$.

*fair headline:
~75× cheaper,
~25× faster (vs
Terra)*

Comparator (workflow mode)	Accuracy vs Jev	Cost multiple	Time multiple
GPT-5.6 Luna	−1.0 pt	~8×	~32×
DeepSeek V4 Flash	−3.4 pts	~15×	~130×
GPT-5.6 Terra (closest match)	+0.1 pt	~76×	~25×
Claude Sonnet 5	0.0 pt	~293×	~195×
GPT-5.6 Sol	+6.3 pts	~209×	~58×
Claude Opus 5	+5.3 pts	~440×	~95×

193.6× sits close to the Sonnet 5 time ratio and 444.6× close to the Opus 5 cost ratio. TypeSafe doesn't say which comparator it used.

Per-workflow accuracy ^(VR)

Workflow	Jev	Best model	GPT-5.6 Terra
Customer service	76.0%	78.3% (Sol)	72.7%
Agent trace observability	71.6%	76.6% (Sol)	73.0%
Security incidents	61.7%	66.2% (Opus 5)	51.2%
Invoice processing (multi-document)	61.8%	79.1% (Sol)	74.7%

Apples to apples?

- Same workflow for every model: yes, a fair design. Every model gains from workflows over one-shot prompts (Haiku 4.5: 18.1% → 53.6%).

- **LLMs in an inefficient shape:** partly. They emit probability distributions through TypeSafe's adapter (with retries for malformed output) at default reasoning; TypeSafe itself says that's slower and costlier. Sonnet 5 takes 78.1s per case. Batching questions or turning reasoning down would shrink the multiples.
- **Output length controlled:** not reported.
- **Capabilities sacrificed:** sums, dates and account checks are computed in code for all models, removing the parts where Jev is weakest.
- **Ground truth:** no, it's frontier-model consensus. The best model reaches only 74.1% agreement, so the ceiling and the reference noise are unknown.
- **Employees involved:** yes, disclosed. TypeSafe says the workflows weren't chosen to favor Jev and sit outside its training distribution.
- **Credit due:** full queries, disagreements and a "nuance" note for every claim is more transparency than most launches. Terra-level agreement at ~1/75 the cost is strong on its own. (0)

Independent comparisons at a glance

Study	Jev	Comparator	Speed / cost
Phishing, 2,000 emails	62.6%	Claude Haiku 4.5: 81.3%	p50 239 vs 687 ms · \$0.038 vs \$0.462 per 1,000
Banking77, n = 208	83.2%	Terra 87.5% · supervised encoder 93.3%	0.44 s vs 1.51 s (Terra)
CLINC150 zero-shot, n = 200	87.0%	Terra 91.5% · gpt-5.4-nano 79.5%	0.43 s vs 1.20 s (Terra)
400-item classification	95.9%	Keywords 77.2% · TF-IDF + LR 66.0%	800 judgments in one call: 985 ms, \$0.00075
6,003 rubric checks (Good Start Labs, early access)	91.5% agreement with Fable 5.1	DeepSeek V4.1 Flash 91.5% · GPT-5.6 Luna 86.5% agreement with Jev	\$160 vs \$33,000 (Fable) and \$260 (DeepSeek) per million verdicts

Good Start Labs had early access. I found no disclosed relationship with TypeSafe for the others.

6 Failure modes

A typed interface eliminates a class of failure without eliminating incorrect predictions. In a closed answer space, hallucination comes back as a confident pick of the least-wrong option.

Removed by construction

- Malformed JSON, parse failures, out-of-schema labels
- Refusals, chatter or off-task text instead of an answer
- Runaway generations and the tail latency they cause
- Invented tool names or categories: Jev can only pick from your list

Still very much possible

```
allowed: fraud = true | false
returned: fraud = false
valid type ✓
correct? unknown
```

TypeSafe's FAQ says the guarantee is about the answer's shape; Jev can still choose the wrong category.

What it does not remove

Failure	Evidence	Mitigation
Confidently wrong, schema-valid answers	Confidence exactly 1.0 on 102/200 CLINC150 items, 6 of them wrong (ickma2311)	Gate plus audit sampling; recalibrate on your labels
Closed-world forcing: the truth isn't among the options	0 of 30 out-of-scope messages flagged, even at a 0.99 gate, without a "none of these" option (priorbench)	Add an explicit none/other option, or a Noul asking whether any option fits
Information missing from state	44.7% accuracy at 0.74 average probability when the deciding policy wasn't in the text (scientheen)	Put the policy in state or apply it in code; ask whether state holds enough to decide
Schema / criteria design errors	Wrong option descriptions: 16.7%, below the 25% chance floor; option order moved results up to 13 points on ambiguous tasks (priorbench)	Treat criteria as code: review, version, regression-test
Miscalibration under shift	ECE ~0.03 on public benchmarks vs 0.107 out of distribution; sign flips by question type	Per-question recalibration; watch the confidence histogram for drift
Structural inconsistency	TypeSafe's own example: the same refund question gave P(yes) 0.22 as a Noul and 0.01 as a Choice; a question and its negation summed to 1.19	Ask each decision one way; enforce identities in code
Math, counting, dates, multi-hop	Listed by TypeSafe as known weaknesses; invoice workflow 61.8% vs 79.1%. One independent test found 99.6% on number and date ordering	Compute in code; ask Jev only the judgment
Adversarial content in state	TypeSafe says jev-1.13 doesn't treat state as hostile	Isolate untrusted text; red-team before deployment
Run-to-run variation	2.2% label flips across passes vs 0.7% for Claude Haiku 4.5 (phishing bench). TypeSafe targets consistency, not determinism	Cache decisions; majority-vote where it matters
Cascading errors	Five independent gates at 95% each all pass only ~77% of the time (0.95^5)	Measure the workflow end to end, not each leaf
No open-ended reasoning or generation	By design	Route those cases to a generative model

Why this matters ⑦

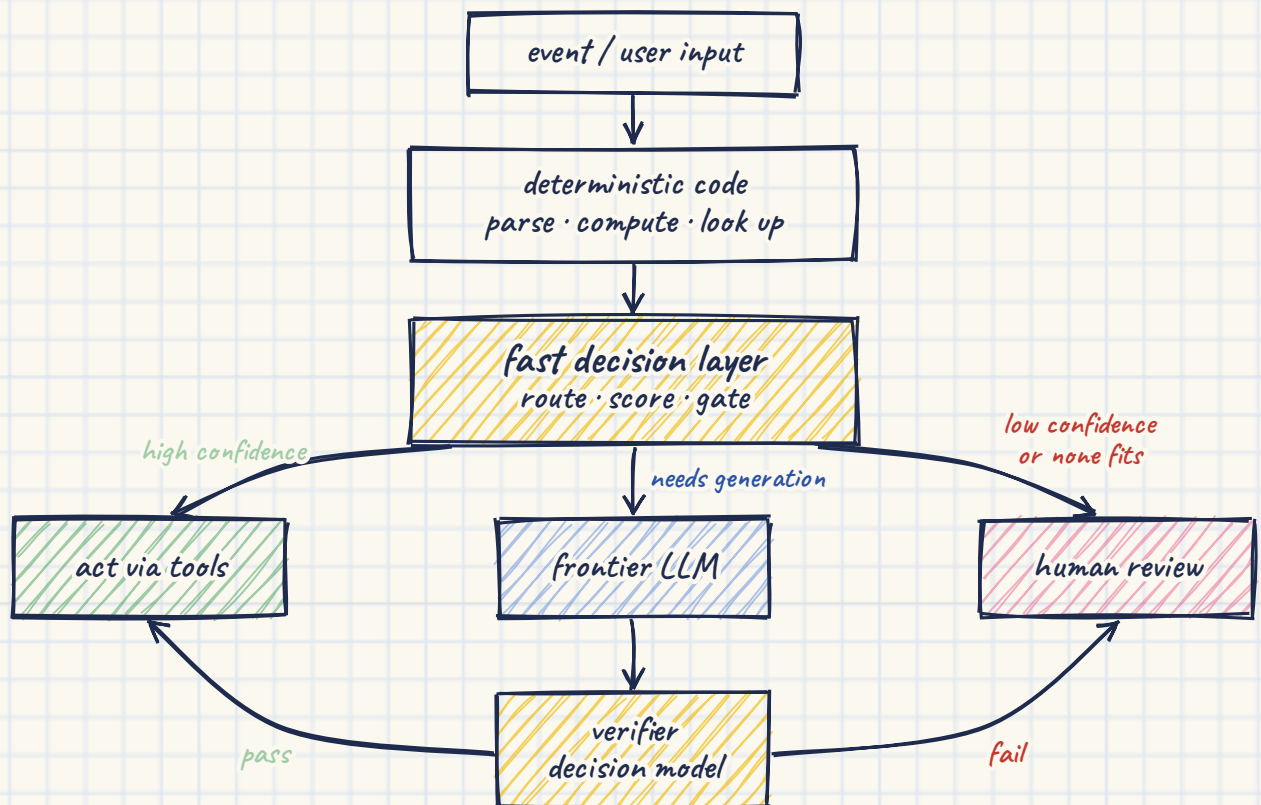
The burden moves to three places:

1. the answer space (does it cover reality, including "none of these"?)
2. the information in state (can the question be decided from what you sent?)
3. the calibration of the threshold (does 0.9 mean 90% on your data?)

With free text, a fabricated answer usually looks fabricated. In a closed set a wrong answer looks exactly like a right one, and there's no rationale to inspect.

⑦ Implications for agents

A Jev-like model fits best as a **gate**: a fast decision layer between cheap code and expensive generation, and again as a verifier after it. It earns its place where a decision is bounded, frequent and latency-sensitive.



the decision layer shows up twice: before the LLM (is it needed?) and after it (is the output OK?)

Where a decision model can sit

Use case	Why it can beat an LLM call here	Watch out for	Evidence
Query / intent routing	Runs before the expensive call, in a few hundred ms; up to 255 labels	Out-of-scope inputs forced into a label	CLINC150: 87.0% vs Terra 91.5%
Tool / skill selection	Choice over tool names plus a "no tool" gate; can't invent a tool	Large catalogs need two stages	TypeSafe cookbook ranks 182 skills, then re-checks the top 3 VR
Safety / guardrail screens	Cheap enough for every message; several hazard questions per pass	Adversarial text is part of state	TypeSafe: jev-1.13 isn't hardened against it VR
Relevance scoring, RAG filtering	One Noul per passage, in parallel	Passages full of irrelevant detail	Top-1 re-ranking 5% → 18% on legal queries VR
Document classification	Deep label trees via repeated Choice	Numeric or code-heavy documents	TypeSafe cookbook, hierarchical beam search VR
Support ticket routing	Queue, urgency and intent from one call	Org rules absent from the ticket	44.7% on a policy-dependent label
Fraud / phishing triage	Many narrow signals, combined in code	The single direct verdict is weak	62.6% direct vs 95.1% via 5 signals + fitted regression
Escalation decisions	Confidence returned natively	Only works if calibrated on your data	100% accuracy above 0.99 on 60.2% of traffic (one test)
Ranking	A Score per item, in parallel	Score levels weakly calibrated as magnitudes	TypeSafe jaggedness page VR
LLM output verification / evals	Makes checking every output affordable	Implied facts, reference contradictions	91.5% agreement with Fable 5.1 at \$160 vs \$33,000 per million verdicts
Workflow branching	Many independent questions, logic in code	Errors compound across gates	$0.95^5 \approx 0.77$

Where the picture is oversimplified

- **Silent errors.** A wrong route has no rationale to inspect; you find it in outcomes or audits.
- **Escalation rate sets the cost.** In a pre-registered cascade test at a 90.5% accuracy target, Jev escalated 22.0% of items vs 48.5% for gpt-5.4-nano, but the confidence interval crossed zero. The author called it ambiguous.
- **Thresholds need labels.** Every gate in the sketch assumes a threshold tuned on data you may not have yet.
- **Supervised small models compete for the same slot.** A frozen encoder + logistic regression hit 93.3% on Banking77 vs Jev's 83.2%, at 0.01 s. Jev's advantage is needing no training data, not beating trained models.

8 Three post angles

Three different arguments the evidence supports, one index card each.

pick

ANGLE A

Constrain the output, and the error moves into the question

<i>Thesis</i>	Jev doesn't remove hallucination; it turns it into closed-world error. The answer space, the state and the threshold now carry the risk.
<i>Evidence</i>	0/30 out-of-scope flagged at a 0.99 gate · 44.7% right at 0.74 probability with policy absent · 16.7% with wrong criteria · TypeSafe's FAQ concedes wrong picks.
<i>Lesson</i>	Design enums like APIs: include "none of these", version criteria, calibrate per question.
<i>Caveat</i>	LLMs with enum schemas share the closed-world problem; it isn't unique to Jev.

ANGLE B

The intelligence is in the decomposition

<i>Thesis</i>	Jev executes leaf judgments well; the workflow an engineer designs does much of the work. A smart if-statement, not a solver.
<i>Evidence</i>	Phishing: 62.6% direct vs 95.1% from 5 signals + fitted regression · every model gains from workflows (Haiku 18.1% to 53.6%).
<i>Lesson</i>	Invest in decomposition and question-level evals; keep math and lookups in code.
<i>Caveat</i>	The 95.1% needed 1,000+ labels and a fitted regression; trained small models did better.

ANGLE C

Price judgment per decision, not per model

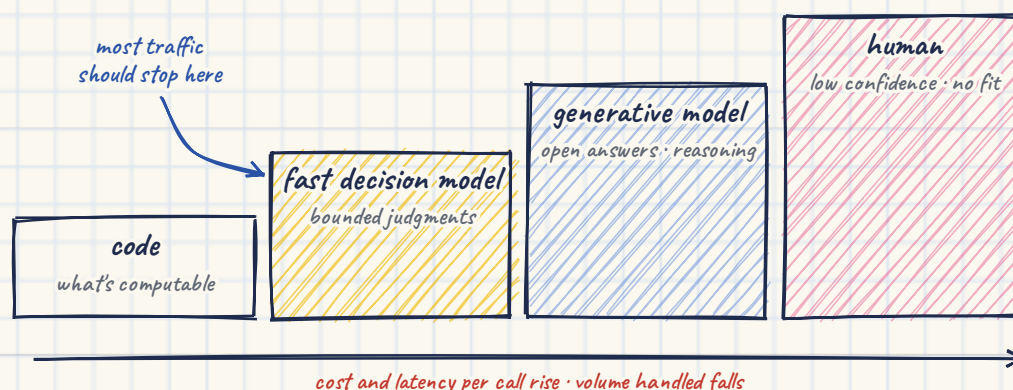
<i>Thesis</i>	Production AI will look tiered: code, fast decision models, generative models, humans, each invoked only when needed.
<i>Evidence</i>	Terra-level agreement at ~1/75 the cost · \$160 vs \$33,000 per million verdicts in Good Start Labs' grading.
<i>Lesson</i>	Build cascades gated by calibrated confidence; track escalation rate as a cost driver.
<i>Caveat</i>	Cascade gains depend on calibration, which is uneven; one pre-registered cascade test was inconclusive.

9 Recommended angle

Angle A, with B as the mechanism and C as the closing lesson.

- It's the least obvious reading of the launch. Most coverage repeats "can't hallucinate"; the systems point is where the error goes instead.
- It's critical without being dismissive. It rests on TypeSafe's own admission plus three independent measurements.
- It generalizes beyond Jev. Any enum-constrained LLM call, classifier or rules engine has the same closed-world failure, so readers can use it on Monday.
- It leads naturally to C, the hierarchy, without claiming that future is certain.

Angle C alone risks reading like a trend post. Angle B alone undersells the model and leans on one benchmark.



the hierarchy the post closes on: every layer is allowed to say "I don't know"

A type system can guarantee a valid answer. It can't guarantee you asked a question the world fits into.

10 The LinkedIn post

Final draft, 849 words with hashtags. Every number in it is checked in section 11. Put the sources in the first comment (card at the end).



Draft · ready to paste

LinkedIn · text post · no images needed

TypeSafe says its new model, Jev, can't hallucinate.

That's true in a narrow sense. The narrow sense is the interesting part.

Jev, launched last week, isn't a chat model. You send it program state (a ticket, a log line, a JSON record) plus typed questions: yes/no, pick one of N options, or a score on a rubric. It returns a probability distribution for each question. No prose, nothing to parse. It cannot return a label you didn't define.

A typical LLM decision path:

state → prompt → reasoning tokens → JSON → parse → validate → branch

Jev's path:

state + typed questions → one parallel pass → probabilities → your thresholds → branch

Why that can be much faster (my reading; TypeSafe hasn't published the architecture): a generative model pays one sequential forward step per output token, and reasoning models emit a lot of tokens before they answer. A closed-set decision needs no decode loop. The answer is a distribution over K known options, available after the input is processed once, and many questions can share that pass. Latency tracks input length, not how long the model decides to talk. It's also why output tokens are free: there's almost no output.

The numbers, read carefully:

- TypeSafe reports 193.6x faster and 444.6x cheaper on its own four-workflow benchmark. The "correct" answers there are the averaged outputs of GPT-6 Astra and Claude Fable 5.1, not human labels, and TypeSafe staff built the workflows. They disclose both, and call these multiples the high end.
- In that eval Jev scores 67.8%, level with GPT-5.6 Terra at 67.9%. From TypeSafe's published per-case figures, that's roughly 75x cheaper and 25x faster than Terra, and roughly 8x cheaper than GPT-5.6 Luna. The multiple depends on the comparator.
- Independent testers measured about 0.24 to 0.5 s per call from different setups, consistent with TypeSafe's 70-500 ms claim.

That's strong economics for a bounded decision. But I think the more important part is this:

Constrain the output, and the error doesn't disappear. It moves into the question.

TypeSafe's own FAQ is candid about it: the guarantee covers the shape of the answer, and Jev can still pick the wrong option. First-week independent tests (single-author GitHub evals, not peer-reviewed) show what that looks like:

1. Closed world. With no "none of these" option, a pre-registered eval found that none of 30 out-of-scope messages were flagged, even at a 0.99 confidence gate. Each one got a valid label.
2. Missing context. On a priority label that depended on an org policy absent from the input, Jev was right 44.7% of the time while putting 0.74 probability, on average, on its pick.
3. Criteria are code. Wrong option descriptions dropped accuracy to 16.7%, below chance for four options. On ambiguous tasks, option order alone moved results by up to 13 points.

Every one of those answers is schema-valid. None is a type error. All are wrong.

Schema validity was never the hard part for bounded decisions anyway: strict structured-output modes on major LLM APIs already constrain decoding to your schema, apart from edge cases like refusals and token limits. What Jev adds is a native probability for every option, many questions per pass, and a price that makes calling it on every event plausible.

That makes calibration the real product claim, and it's the least documented one. I couldn't find a calibration curve or ECE published by TypeSafe. One independent study found good calibration on public benchmarks (ECE around 0.03) but 0.107 on out-of-distribution tickets: overconfident on multiple choice, underconfident on yes/no.

Decomposition carries a lot of weight too. On 2,000 phishing emails, a direct "phishing or legitimate?" question scored 62.6% (Claude Haiku 4.5: 81.3%). Five narrow signal questions from the same call, combined by a logistic regression fitted on labelled emails, reached 95.1%. Part of that is Jev. Part is the engineer who chose the signals, and part is the labels.

What I'd take from this:

- The enum is the prompt. Review, version and test it like code.
- "None of the above" is a safety mechanism, not a nice-to-have.
- Treat confidence as a ranking signal until you've calibrated it on your own labels, per question type.
- Keep arithmetic, dates and lookups in code. TypeSafe's own docs say the same.
- Measure the workflow, not the leaf. Five gates at 95% each, with independent errors, all pass only about 77% of the time.

The bigger idea isn't one model cheap enough to call everywhere. It's a hierarchy where every layer is allowed to say "I don't know":

code for what's computable

- a fast decision model for bounded judgments
- a generative model when the answer can't be enumerated or needs reasoning
- a human when confidence is low or no option fits

Jev gives up generation, explanation and open-ended reasoning to be one fast layer in that stack. That's specialization, not general intelligence, and for the right workloads it's a good trade.

A type system can guarantee a valid answer. It can't guarantee you asked a question the world fits into.

Sources and the independent evals are in the first comment.

#AIEngineering #LLM #MLSystems #AgenticAI

First comment paste right after posting

Sources (primary first):

TypeSafe launch post: <https://typesafe.ai/blog/introducing-system-one-models-and-jev>

TypeSafe workflow evals: <https://evals.typesafe.ai/>

Jev known failure modes (TypeSafe docs): <https://docs.typesafe.ai/model-jaggedness/jev-1.13.md>

Independent evals (single-author, not peer-reviewed):

Pre-registered eval: <https://github.com/priorbench/jev>

Out-of-distribution calibration: <https://github.com/scienthoon/jev-ood-calibration>

Phishing benchmark vs Claude Haiku 4.5: <https://github.com/anisselbd/jev-phishing-bench>

Banking77 / CLINC150 baselines: <https://github.com/ickma2311/jev-baselines-eval>

11 Fact-check table

Every factual claim in the post, checked against pages opened on 21 Sep 2026.

Verified primary source or independent measurement

Vendor-reported TypeSafe's own claim

Inference my reasoning or arithmetic

Opinion judgment

Claim in the post	Evidence (source)	Status	Confidence
TypeSafe says Jev can't hallucinate	Launch post (primary)	Verified that they say it	High
Launched last week	Launch post dated 15 Sep 2026 (primary)	Verified	High
State + yes/no, pick-one-of-N or rubric questions	API reference, primitives docs (primary)	Verified	High
Can't return a label you didn't define	By construction; 0 invalid responses in 4,621 independent calls	Verified	High
No decode loop explains the speed; latency tracks input length	Architecture unpublished; consistent with input-only pricing and "parallel" description	Inference	Medium
Output tokens are free	Models page: output not billed (primary)	Verified	High
193.6× faster, 444.6× cheaper	Homepage, traced to workflow evals (primary)	Vendor-reported	High it's their claim; not reproduced
Reference = averaged GPT-6 Astra & Fable 5.1 outputs	Evals site + launch post (primary)	Verified	High
Staff built the workflows; multiples called "high end"	Launch post nuance notes (primary)	Verified	High
Jev 67.8% vs GPT-5.6 Terra 67.9%	Evals overview (primary)	Vendor-reported	High
~75× cheaper, ~25× faster than Terra; ~8× cheaper than Luna	Arithmetic on published per-case means (rounded)	Inference	Medium (±10–15%)
Independent latency ~0.24–0.5 s per call	239 ms p50 (phishing bench); ~430 ms floor (priorbench); ~0.43 s (ickma2311); ~0.5 s (Good Start Labs)	Verified single-author setups	Medium-high
TypeSafe claims 70–500 ms	Launch post table (primary)	Vendor-reported	High
FAQ: guarantee covers shape; Jev can pick the wrong option	Homepage FAQ, "Can Jev still get things wrong?" (primary)	Verified	High
0 of 30 out-of-scope flagged at a 0.99 gate, no "none of these"	priorbench/jev README	Verified one pre-registered study	Medium
44.7% accuracy at 0.74 avg probability, policy absent	jev-ood-calibration	Verified one study, synthetic data	Medium
Wrong criteria 16.7% (below 25% chance); option order up to 13 pts	priorbench/jev	Verified one study	Medium
Strict structured outputs constrain decoding to the schema, except refusals and token limits	OpenAI, 6 Aug 2024 (primary, other vendor)	Verified	High
No calibration curve or ECE published by TypeSafe	Absent from launch post, confidence docs, AI primer; also noted by Cherry Creek News	Verified as of 21 Sep 2026	Medium-high

Claim in the post	Evidence (source)	Status	Confidence
ECE ~0.03 public benchmarks, 0.107 OOD; Choice over-, Noul underconfident	jev-ood-calibration	Verified one study	Medium
Phishing 62.6% vs Haiku 4.5 81.3%; 5 signals + fitted LR 95.1%	jev-phishing-bench	Verified synthetic bodies, URL-reputation labels	Medium
TypeSafe docs say keep arithmetic and dates in code	jev-1.13 jaggedness page (primary)	Verified	High
Five 95% gates all pass ~77% of the time	$0.95^5 = 0.774$, independent errors assumed	Inference	High (arithmetic)
Jev gives up generation, explanation, open-ended reasoning	System One concept page (primary)	Verified	High
Hierarchy: code, decision model, LLM, human	Supported but not proven by the evidence above	Opinion	—

Deliberately left out of the post: the \$40M seed round (secondary sources only), and "co-inventor of RLHF" (TypeSafe's framing; Almeida is a verified InstructGPT co-author, but RLHF predates that paper).

12 Sources

Opened on 21 Sep 2026 unless noted. Links are clickable.

TypeSafe (primary)

- [Introducing System One Models & Jev](#), Diogo Almeida, 15 Sep 2026
- [TypeSafe homepage and FAQ](#)
- [Workflow evals · Invoice Processing workflow](#)
- [Docs: Introduction · Quick start · System One · API reference · Models · Confidence · AI primer · Score · Noul](#)
- [Jev 1.13 jaggedness \(known failure modes\)](#)
- [System One adapter used for the LLM baselines](#)

Independent evaluations *(single-author, not peer-reviewed)*

- [priorbench/jev](#): pre-registered, 5,721 calls, 21 experiments, 20 Sep
- [scienthoon/jev-ood-calibration](#): calibration in and out of distribution, 19 Sep
- [anisselbd/jev-phishing-bench](#): 2,000 emails vs Claude Haiku 4.5, 17 Sep
- [ickma2311/jev-baselines-eval](#): Banking77 and CLINC150 vs nano, frontier and supervised baselines, 18 Sep
- [Good Start Labs: Verification is the bottleneck](#): 6,003 rubric checks (early access)

Coverage & analysis (secondary)

- [TechCrunch](#), 18 Sep 2026
- [Langfuse: Using TypeSafe's Jev for evals](#)
- [The Cherry Creek News: eval methodology critique](#)
- [XenoSpectrum: Jev vs BERT-style classifiers](#)
- [THE D*AI*LY BRIEF: decomposition and calibration roundup](#)

Background

- [OpenAI: Introducing Structured Outputs in the API](#), 6 Aug 2024
- [GPT-4 Technical Report](#), Fig. 8 on calibration before and after post-training
- [Guo et al., On Calibration of Modern Neural Networks](#), 2017 (cited from knowledge, not re-opened)
- [Ouyang et al., Training language models to follow instructions with human feedback](#) (InstructGPT author list)

— end of notes —